Prep notes: AI-powered support ticket system (FTR-0)

Internal preparation only. This document is for our group to get ready before we present to the instructor. Do not submit it or share it outside the team. The deck we present is `G7_46-60-88-101_FTR0.pptx`. These notes exist so each of us can explain the project naturally instead of reading from the slides.

The pitch (memorize this)

We are building a helpdesk system that reads incoming support tickets with our own small AI model, tags each one by category and urgency, sends it to the right agent, and watches SLA deadlines with live timers. The model runs on our own servers, so ticket data never goes to an outside API and there are no per-ticket fees.

If you say only one thing, say this: **private, cheap, and self-improving, because the AI is in-house.**

The problem we solve

Small and medium companies still sort support tickets by hand. Four things go wrong:

1. **Slow first response.** Tickets sit in a shared queue until someone reads them.
2. **Misrouting.** Tickets land with the wrong team and bounce between departments ("ticket ping-pong").
3. **SLA breaches.** No live timers, so a missed deadline is discovered after it happens. (SLA = service level agreement, the promised response and resolution time.)
4. **Agent overload.** Work spreads unevenly and agents retype similar replies all day.

Commercial helpdesks like Freshdesk and Zoho Desk automate some of this, but they charge per agent per month, and their AI features send ticket text to external cloud APIs. Many organizations cannot accept confidential support data leaving their premises. That cost and privacy gap is our opening.

Our idea in four questions

- **What:** replace manual, error-prone triage with automated classification, routing, and tracking.
- **Why:** SLA breaches, high cost, and data-privacy risk.

- **How:** a small language model (SLM) fine-tuned in-house, inside a full web system (React + FastAPI + PostgreSQL).
- **Who:** SMEs, their support agents and managers, and the customers who raise tickets.

The AI never acts alone. It suggests a category, a priority, a route, and a draft reply, and a human agent approves or corrects all of it. Every correction is saved and later used to retrain the model, so it keeps improving on our organization's real tickets.

How the system works (tell it as a story)

1. A customer submits a ticket, with attachments, through the web form.
2. The FastAPI backend stores it, writes an audit log entry, and starts the SLA timer.
3. The ticket text goes to our SLM service, which predicts category and priority and drafts a reply.
4. The routing engine assigns the ticket to an agent using the predicted category, the agent's current workload, and how much SLA time is left.
5. The agent's dashboard updates instantly over WebSockets.
6. The agent reviews the AI suggestions, fixes anything wrong, and sends the reply. Corrections go into a feedback table.
7. If a deadline gets close or is missed, the system escalates and sends email or Telegram alerts.
8. On a schedule, the saved corrections are used to fine-tune the model again (LoRA), and the updated weights go live.

Small glossary (one line each)

- **SLM (small language model):** a compact model such as DistilBERT or Qwen2.5-0.5B that runs on normal hardware.
- **Fine-tuning:** training a pretrained model further on our ticket data so it learns our categories.
- **LoRA:** a cheap fine-tuning method that trains a few small added weights instead of the whole model, light enough to run on Colab.
- **Inference:** running the trained model to get a prediction.
- **Macro-F1:** the F1 score averaged equally over all classes, so rare ticket types count as much as common ones.
- **REST API:** the backend endpoints the frontend calls to create, fetch, and update tickets.
- **JWT:** a signed token that proves who is logged in; we use it for login and role checks.
- **RBAC:** role-based access control; customers, agents, and admins see different things.
- **WebSockets:** a live connection that pushes dashboard updates without page refresh.
- **CRON / scheduled task:** a job that runs automatically at set times, for example the retraining job and SLA checks.

Literature review in one breath

We reviewed 8 papers from 2022 to 2026, plus commercial tools. Short version: machine learning can classify tickets well, but published work stops at classification. Three examples worth remembering:

- **Ticket-BERT (2023):** a fine-tuned BERT reached macro-F1 around 98% on Microsoft Azure incident tickets, but it is tied to Azure data and does no routing or SLA work.
- **Razma and Jurkus (2026):** on imbalanced multilingual enterprise data, mBERT reached macro-F1 only 0.51 against 0.39 for TF-IDF with Logistic Regression. Lesson: transformers help, but class imbalance hurts everyone, which is why we keep a classical baseline for comparison.
- **Crosley and Wasito (2023):** IndoBERT plus Random Forest on one company's tickets worked, but a single dataset and a coarse priority scheme limit it.

Common limitations across the papers: single-company datasets, models trained once and then frozen, no routing or SLA or feedback integration, and results reported as plain accuracy.

Gaps mapped to our objectives (your safety net)

Gap in existing work	Our objective
AI features call external cloud APIs, so data leaves the premises	Fine-tune an in-house SLM; everything runs on-premise
Models are frozen after training and never learn from corrections	Feedback loop: agent edits go into the dataset for periodic retraining
Research stops at classification; no SLA, audit, or access control	One system with SLA timers, escalation alerts, audit logs, and RBAC
SaaS pricing plus AI add-ons is too costly for SMEs	Open-source stack with no per-ticket fees
Classifiers ignore agent skill, workload, and remaining SLA time	Routing engine combines the prediction with workload and the SLA clock

Numbers to remember

- Classification: **macro-F1 of 0.85 or higher**, against a roughly **72%** TF-IDF + Logistic Regression baseline.
- Routing: **80% or more** of tickets reach the right agent on the first assignment.
- SLA compliance: **15 to 20% improvement** over the manual baseline.
- Latency: **under 5 seconds** per ticket on CPU, with no external API call.
- Evaluation data: a held-out set of anonymized historical tickets from our industry partner.
- Delivery target: **31 August 2026**, in four phases: foundation and basic system, then AI classification with routing and draft replies, then SLA monitoring with the feedback loop and alerts, then full integration, testing, and delivery.

Scope, if someone asks

In scope: the full ticket lifecycle, AI category and priority tagging, automated routing, AI-drafted replies with mandatory agent approval, feedback-driven retraining, SLA timers and alerts, a live dashboard, RBAC, audit logs, and Dockerized deployment.

Out of scope for now: voice and social-media channels (web form and email only), large external LLMs or paid AI APIs, native mobile apps, multilingual support (a phase-2 idea), and fully automatic replies with no human check.

Team responsibilities

Member 1: Frontend (React UI and dashboards)

One line: "I build everything the customer and the agent see and click."

Role: the user and agent interfaces.

Tasks:

- Create the ticket submission form.
- Build the agent dashboard with open tickets and SLA countdowns.
- Design the screen where agents review and correct the AI's predicted category.

Deliverables: a React UI (Material-UI or Bootstrap) integrated with the backend API.

Terms to be ready to explain: API integration, React state management, UI routing.

Member 2: Backend (FastAPI and PostgreSQL)

One line: "I build the server and the database that everything else talks to."

Role: the backend and data layer.

Tasks:

- Set up PostgreSQL with Users, Tickets, and Logs tables.
- Build FastAPI endpoints to create, fetch, and update tickets.
- Implement login and role-based access.

Deliverables: a REST API and a relational database.

Terms: relational database schema, RESTful API, basic JWT authentication.

Member 3: AI model trainer (Python and Colab)

One line: "I teach the model to read a ticket and say what it is and how urgent it is."

Role: the ticket classification model.

Tasks:

- Fine-tune a lightweight model (for example DistilBERT) on a CSV dataset of IT tickets.
- Predict ticket category and urgency.
- Save agent corrections for future retraining.

Deliverables: a trained model file (.bin or .pt) and an inference script.

Terms: NLP text classification, fine-tuning, dataset preprocessing, inference.

Member 4: Logic and deployment

One line: "I connect the pieces, run the SLA clock, and put the system online."

Role: integration, SLA logic, and deployment.

Tasks:

- Implement the SLA timer logic.
- Connect the AI service to the FastAPI backend.
- Deploy the project to a free or student cloud platform (Render, Railway, or Heroku).

Deliverables: the SLA alert system, the working end-to-end integration, and a live deployment URL.

Terms: cloud deployment, scheduled tasks (CRON), system integration, SLA logic.

Questions the instructor may ask

- **Why not just call ChatGPT or another LLM API?** Cost per ticket, latency, and privacy. Confidential ticket text would leave the organization, and avoiding that is the whole point of the project.
- **Can a small model really do this?** Ticket classification is a narrow task. A small model fine-tuned on domain data can beat a generic large one here, and our 0.85 macro-F1 target sits between the 72% classical baseline and published BERT results.
- **What if the model is wrong?** An agent approves every classification and every draft reply before anything reaches the customer. Wrong outputs get corrected, and the corrections become new training data.
- **Where does the training data come from?** Anonymized historical tickets from our industry partner, plus the corrections collected while the system runs.

- **Why macro-F1 instead of accuracy?** Ticket classes are imbalanced. Accuracy can look high while the model fails on rare but critical categories. Macro-F1 weights every class equally.
- **How will you retrain cheaply?** LoRA fine-tuning on Colab. Only small adapter weights are trained, so modest GPU time is enough.
- **How do you measure success?** On the held-out ticket set: macro-F1 and per-class recall for classification, routing checked against historical human assignments, and SLA breach rate before versus after automation.

Presenting tips

- Know the pitch and the five numbers cold. Everything else you can talk around.
- Explain your section as the story above, in your own words, not as slide bullets.
- If you blank, come back to the anchor: in-house model, data stays private, near-zero cost per ticket, improves from agent feedback.
- Hand over by name: "and [name] will now take you through the next part."